

UVM Final Project(PCDA Training)

Su Lin Poh

Final UVM Project: What you will learn?

- Apply the knowledge and skills learnt through the training on the project
- Brush up techniques for a SMART Testplan
- Testbench Planning
- Writing Sequences/Tests
- Checkers and Assertions
- Testbench/ Design Debug

LED MUX Controller.

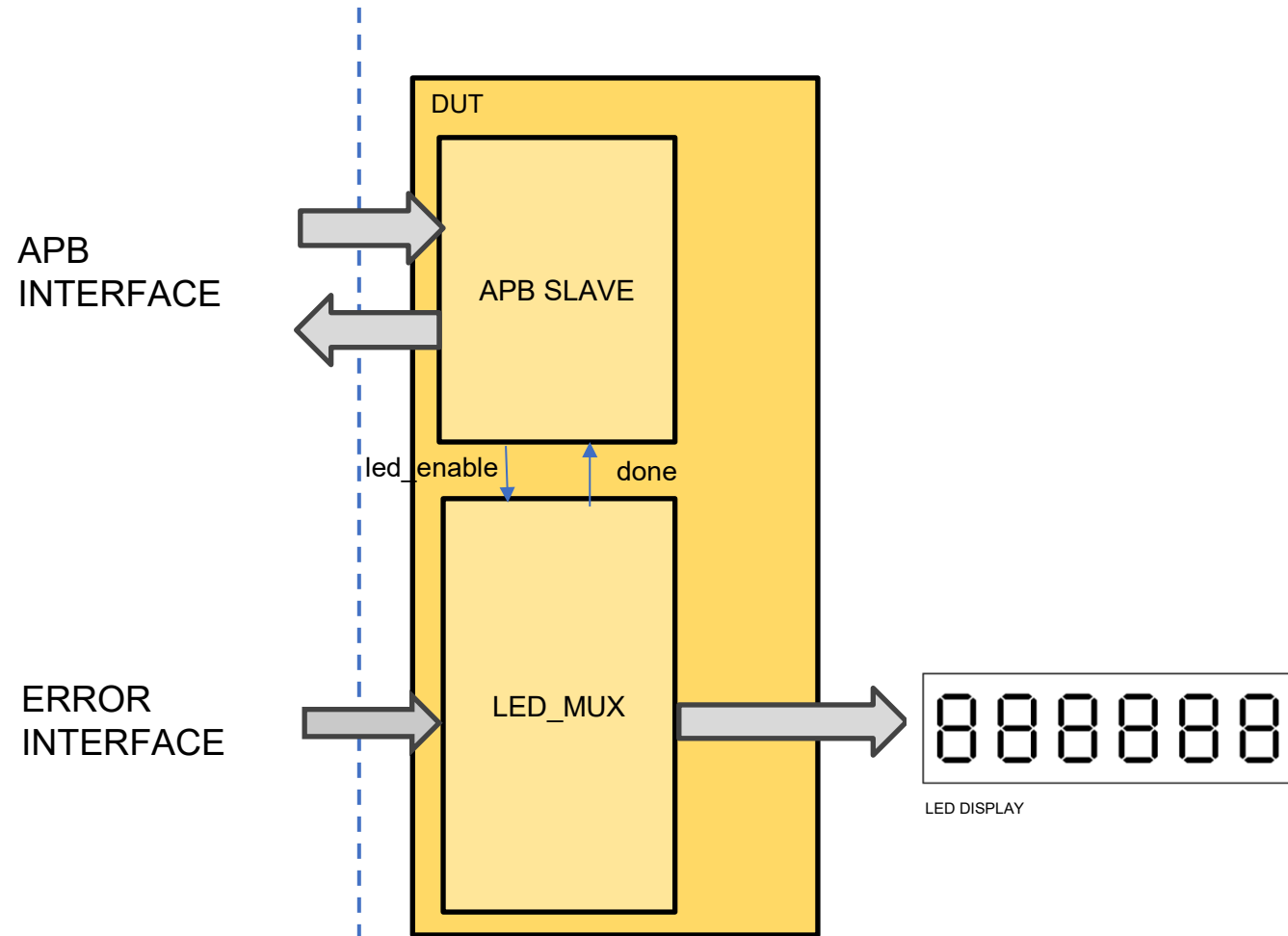


Figure: DUT diagram

Interface (LED INTERFACE)

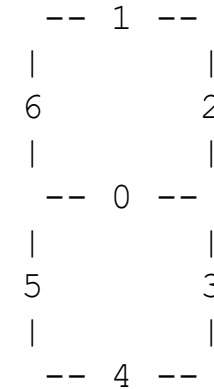
Signal Name	Direction	Width(no of bits)	Description
clk	Input	1	System clock signal (50 MHz)
rst_n	Input	1	Active-low reset signal
error_q	Input	20	No of errors in Hex. Maximum number is 2^{20}
sel_out	Output	6	One Hot Select signal for each digit of LED Display. Active Low
seg_out	Output	8	segment enable for each LED Letters. Active Low except bit[7].

APB Interface

Signal Name	Direction	Width(no of bits)	Description
i_paddr	Input	32	Address bus, specifies the address of the transfer
i_psel	Input	1	Select signal, indicates the slave is selected
i_penable	Input	1	Enable signal, indicates the second and subsequent cycles of an APB transfer
i_pwrite	Input	1	Write control signal, high for write, low for read
i_pwdata	Input	32	Write data bus, carries data to be written to the slave
o_prdata	Output	32	Read data bus, carries data from the slave to the master
o_pready	Output	1	Ready signal, indicates slave is ready to complete the transfer
o_pslerr	Output	1	Slave error signal, indicates an error during transfer

Functional Description(LED Display Mux Controller)

- LED Display Mux controller converts the number of errors from binary to decimal, and display it out on the 6 LED lights. The error value needs to be maintained for a refresh rate of 6 cycles, so that the information will be propagated to all 6 LED lights. Sel_out is a low active signal. seg_out[6:0] is also an active low signal. 0-enable, 1-disable. seg_out[7] is always 1 when the segments are active.
- On top of the 6 cycles, for human visibility, the values should maintain for at least 2 seconds. (20μs for simulation purposes) . E.g the clk input of the mux is 50Mhz, the error should be maintained for at least 100 Million cycles(1000 cycles for simulation purposes, choose 1002 cycles because its divisible by 6). It takes at least 60-80 cycles from input for changes to take effect at output of the mux controller.
- At reset, sel_out are expected to be 6'h3E, and seg_out should have the value 8'h80.



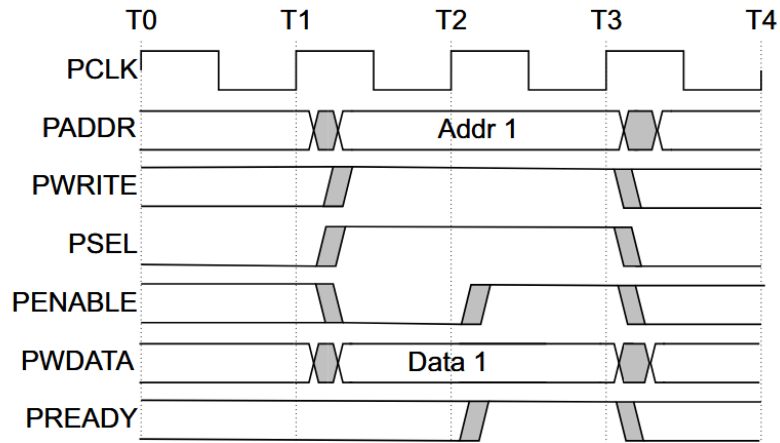
Segment Mapping for Digits

Digit	Segments On
0	1, 2, 3, 4, 5, 6
1	2,3
2	0, 1, 2, 4, 5
3	0, 1, 2, 3, 4
4	0, 2 ,3, 6
5	0, 1, 3, 4, 6
6	0, 3, 4, 5, 6
7	1, 2, 3
8	all segments
9	0, 1, 2, 3, 6

APB Block

- Writes to Register Space to enable the LED Controller. By default, Led Enable is set to 1.
- APB Interface allows CPU to read status back from the LED Controller.
- APB Control also has a 32-bit data scratch pad(PADDR=32'h4008 for CPU to write and read from this interface.
- See the next table for address.

APB Write Cycle



APB READ CYCLE

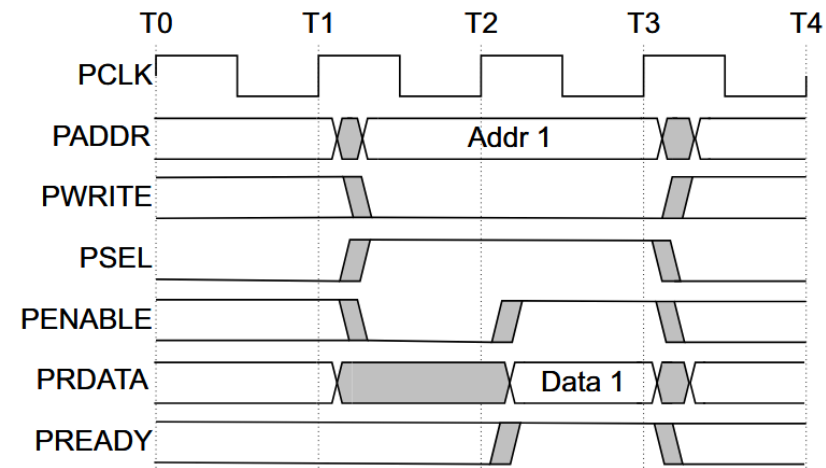


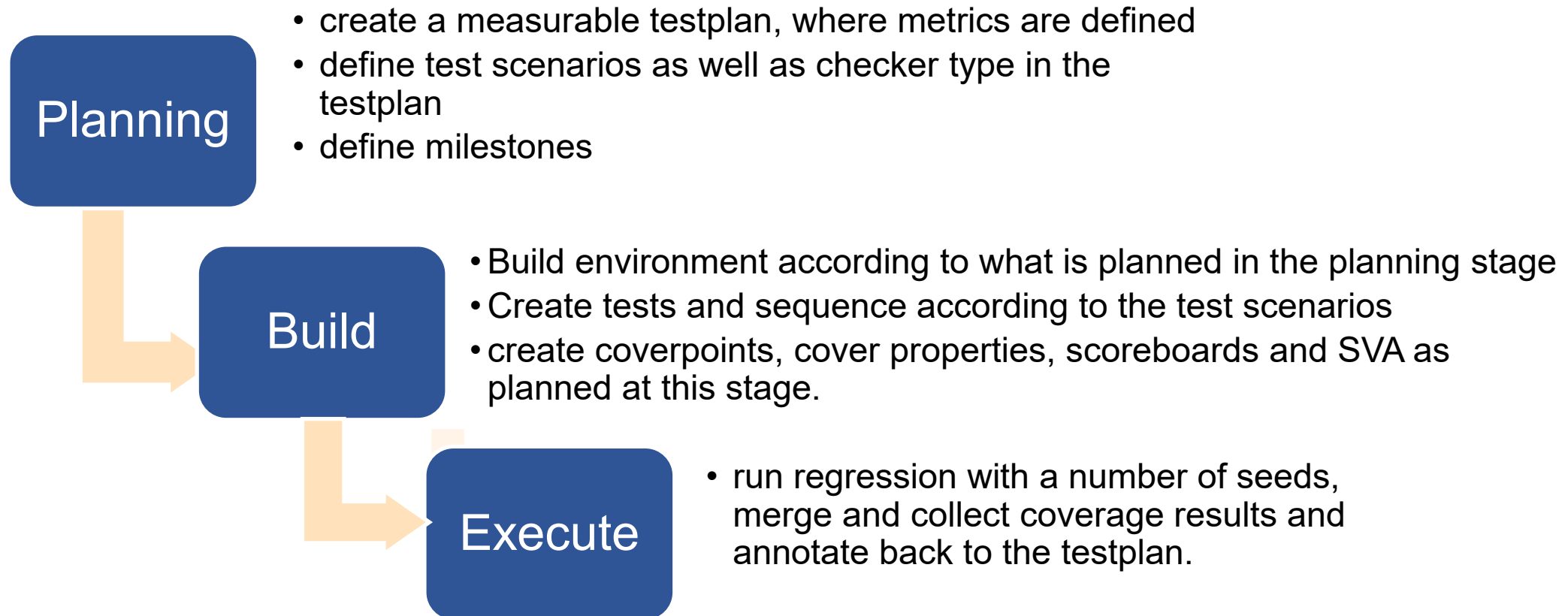
Figure 3-4 Read transfer with no wait states

Register Fields

Register[Addr]	FIELD Description	Description
Control (16'h4000)	Bit[0]/RW-> LED_enable	Set to enable to propagate error_sig value to output seg_out[7:0]. By Default its set to 1.
Status 1 (16'h4004)	Bit[0]/RO- Done	If Done bit is 1, seg_out[7:0] should mapped value match with the error_q. When Done bit is 0. seg_out value is underterministic. Default value is 0.
Scratch Pad (16'h4008)	Bit[31:0]/RW	Write anything to this address, expects read back to be the same value. Default value is 0

Drafting a Testplan

Metrics Driven Verification



S.M.A.R.T Testplan

Specific/ Scope

- clearly define the specific feature set for your DUT.

Measurable

- Quantity that can be measured

Achievable

- Realistic Goal

Relevant

- making sure testing objectives are relevant to the main functionality/goal of your overall project.

Time-bound

- Set milestone for your projects.
- making sure the timelines are set for each the test features

Sample Format for a Testplan

feature set

Testplan Name

References to other testplan

hvp plan <Project_name>	feature	feature	\$owner	\$at_least	\$description	value Assertions. Assert	value Func_Cov. Group	value Code_Cov. Line	value Code_Cov. Cond	value Code_Cov. Toggle	value Code_Cov. FSM	value Code_Cov. Branch	value Code_Cov. SnpsAvg	value TestPlan. test	measure Assertions. source	measure Func_Cov. source	measure Code_Cov. source	measure TestPlan. source	include
plan			<owner>																<Testplan.xml>
	Testplan_summary		<owner>																<Coverageplan.xml>
		subplan <TP_Sheet_1>																	<Assertionplan.xml>
		subplan <TP_Sheet_2>																	
		subplan <TP_Sheet_3>																	
		subplan <TP_Sheet_4>																	
		subplan <TP_Sheet_5>																	
	Code_Coverage		<owner>														tree: <tb_top. rti_inst>		
	Functional_Coverage		<owner>	<No_of_Covergroups>															
		subplan <Coverage_Plan>																	
	Assertionplan_summary		<owner>												<tb_top. assert_inst>				
		subplan <Assertion_Plan>																	

Mapping to the database

Figure 4. Generalize Format of Top Level Verification Plan (Kalavadiya & Patel, 2024)

Feature and Sub-feature

- Split the objectives into main features
 - Based on block division
 - Based on main feature set
- Split main features into sub-features
 - measurable
 - achievable
 - time-bound.
- Does not matter even if you have to split into sub^N features

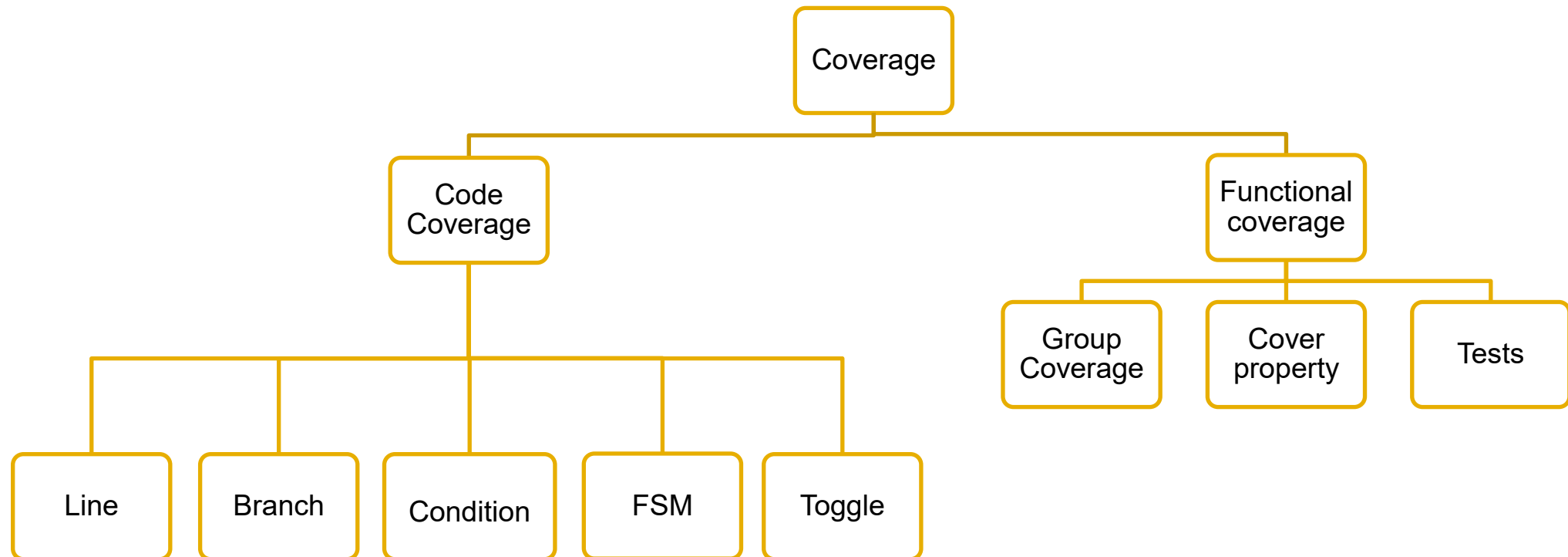
Owner name and Description

- Always define an owner for your feature set
 - Set accountability, a record.
- Description should include details such as :
 - What to be measured?
 - How we can achieve it?
 - Testing Procedures
 - Passing Criteria
 - Exception

Feature set to map to simulation database

- Tests
- covergroups/coverpoints
- cover property
- System Verilog Assertions(SVA)

Types of coverage

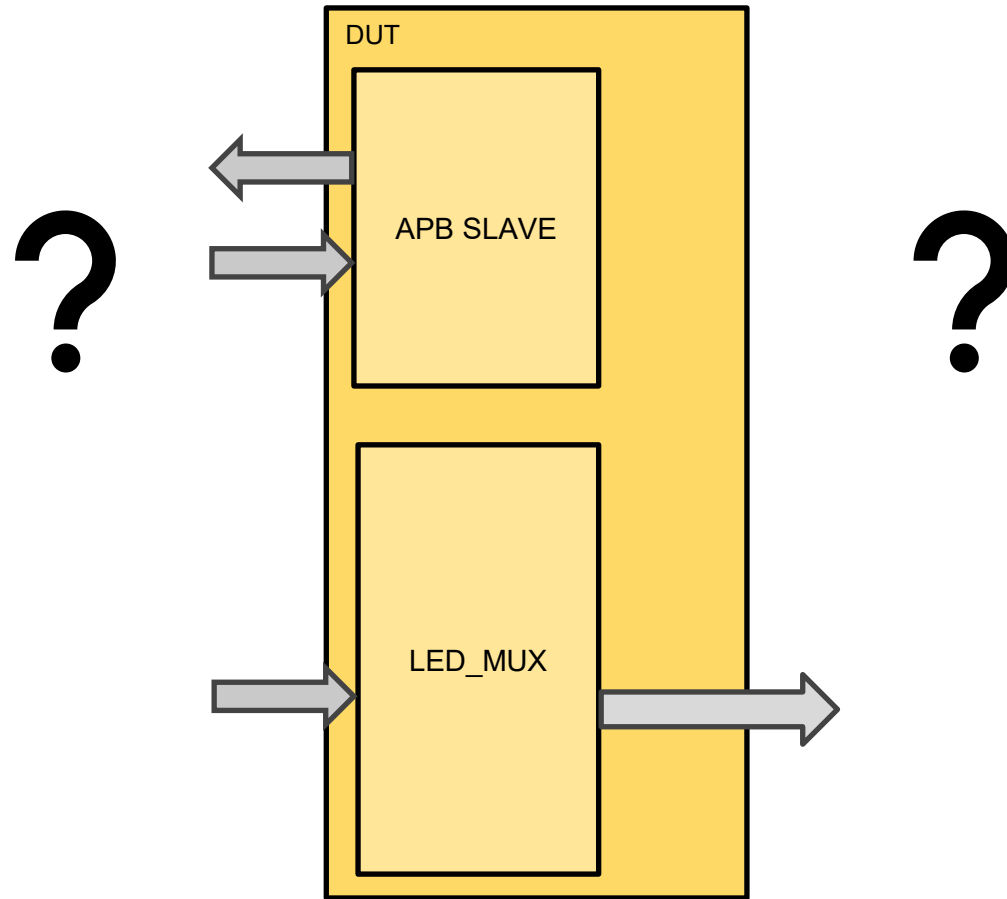


Plan Your Testbench

Things to consider

- Clock and Reset Requirements
- The interfaces
- External component needed to make your design work
- What kind of stimulus is needed? and how to drive those stimulus
- What should be tracked? how should you track it?
- What simulation control should you need?

Testbench Diagram(FILL UP)



Writing A Sequence

Example code for sequence.

```
class my_sequence extends uvm_sequence#(my_transaction);

  `uvm_object_utils(my_sequence)

  function new (string name = "");
    super.new(name);
  endfunction

  task body;
    repeat(8) begin
      req = my_transaction::type_id::create("req");
      start_item(req);

      if (!req.randomize()) begin
        `uvm_error("MY_SEQUENCE", "Randomize failed.");
      end

      finish_item(req);
      get_response(rsp);
    end
  endtask: body

endclass: my_sequence
```

1) waits for grant

1) sends item to the sequencer's queue
2) Wait_for_item_done()

Sample Code: To start a sequence in a test

```
class my_test extends base_test;
`uvm_component_utils(my_test)

//create a handle for your seq:
dp_base_sequence myseq;

function new(string name, uvm_component parent);
super.new(name, parent);
endfunction //new

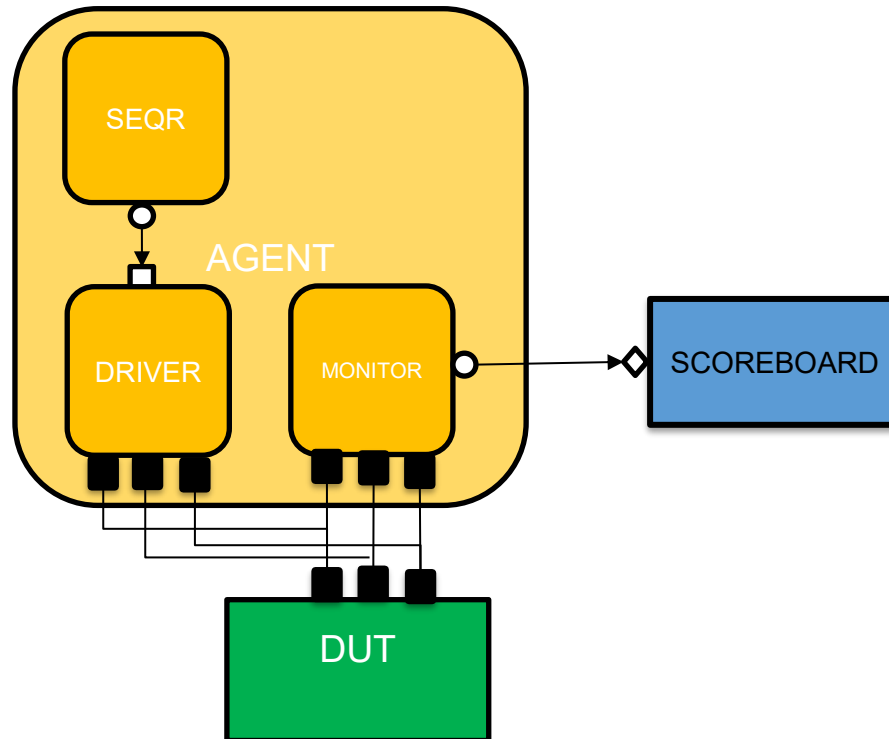
task run_phase(uvm_phase phase);
// Set the action for UVM_ERROR to UVM_DISPLAY and UVM_LOG
// We raise objection to keep the test from completing
phase.raise_objection(this);

`uvm_info(get_type_name(), "is running!",UVM_LOW)
//start a sequence here: e/g:
myseq=dp_base_sequence::type_id::create("myseq",this);
myseq.randomize() with {iteration == 1};
myseq.start(env.agent1.sequencer);// point to the slave agent.

phase.drop_objection(this);
phase.phase_done.set_drain_time(this,10000ns);
endtask //run_phase
endclass
```

Checkers and Assertions

uvm_scoreboard



- is a verification component that contains checkers and verifies a functionality
- usually receives the transactions which are captured by monitors
- May contain one or more analysis exports.
- May receive from one or many broadcasters(Monitors)
- A complex scoreboard may contain reference models

Sample Scoreboard Class

UVM provides us with **uvm_macros** to declare two or more implementation ports in a single scoreboard

```
`uvm_analysis_imp_decl(_f2s_s2f)
`uvm_analysis_imp_decl(_spi1)

class env_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(env_scoreboard)
```

Sample Scoreboard

```
class env_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(env_scoreboard)

  uvm_analysis_imp_f2s_s2f#(spi_transaction,env_scoreboard) aep_f2s_s2f;
  //analysis_export
  uvm_analysis_imp_spi1#(spi_transaction,env_scoreboard) aep_spi1;

  function void build_phase(uvm_phase phase);
    aep_f2s_s2f= new("analysis_port_spi0", this);
    aep_spi1= new("analysis_port_spi1", this);
  endfunction
```

```
function void write_f2s_s2f(spi_transaction TR);
  spi_transaction local_tr;
  $cast(local_tr, TR.clone());
  //implement checker here
  ...
endfunction
function void write_spi1(spi_transaction TR);
  ...
endfunction
```

Add the scoreboard in the ENV

- Instantiate the scoreboard
- New the scoreboard(by calling the factory create() function)

```
class my_env extends uvm_env;
  `uvm_component_utils(my_env)

...
  env_scoreboard      scoreboard1;
  ...
  function void build_phase(uvm_phase phase);
    scoreboard1=env_scoreboard::type_id::create("scoreboard1",this);
  endfunction
  ...
```

The scoreboard and monitor connection

```
class my_env extends uvm_env;
```

```
...
```

```
function void connect_phase(uvm_phase phase);
```

```
    agent.monitor.analysis_port.connect(scoreboard1.aep_f2s_s2f);
```

```
endfunction
```

Do it in the **connect()** phase

PRESENTATION

- Objective
- Testplan Overview
- Summary of your effort
- What are your findings? Conclusion. Any Future Improvements?

Summary

Day 1- Recap, Testplan Writing

Day 2- Create Testbench, Writing Tests/Sequences.

Day 3- Scoreboarding + Checkers (SVA)

Day 4- Testplan Execution, Virtual Sequence(Stretch Goal)

Day 5- Coverage, Regression and Wrap-up

Day 6- PRESENTATION DAY

“Kindness you receive today is not incidental, pay it forward when you can” – Su Lin–

Thank You

References

Chris Spear(2008) *SystemVerilog for Verification. 2nd Edition*. MA,USA: Springer.

Accellera Systems Initiative (2012) *UVM Class Reference Manual 1.2* [Preprint]. Available at: https://www.accellera.org/images/downloads/standards/uvm/UVM_Class_Reference_Manual_1.2.pdf.

Accellera Systems Initiative (2012) *UVM Users Guide 1.2* [Preprint]. Available at: https://www.accellera.org/images/downloads/standards/uvm/uvm_users_guide_1.2.pdf (Accessed: 13 February 2024).

Patel, S. (no date) *System Verilog assertions simplified, Design And Reuse*. Available at: <https://www.design-reuse.com/articles/44987/system-verilog-assertions-simplified.html> (Accessed: 13 February 2024).

Angelo Jacobo– FPGA SDRAM CONTROLLER Repository. GitHub.
https://github.com/AngeloJacob/FPGA_SDRAM_Controller(Accessed: 03 September 2025).